In both cases, you need to point at the block and say, "I want to use *this* block"—to do that, you need a name. To attach a binding to the block, you can add one special argument to the method. This argument must be the last in the list of arguments and prefixed by an & sign. Here's a method that passes the block to another method:

blocks/ampersand.rb

```
def math(a, b)
  yield(a, b)
end

def do_math(a, b, &operation)
  math(a, b, &operation)
end

do_math(2, 3) {|x, y| x * y}  # => 6
```

If you call do_math without a block, the &operation argument is bound to nil, and the yield operation in math fails.

What if you want to convert the block to a Proc? As it turns out, if you referenced operation in the previous code, you'd already have a Proc object. The real meaning of the & is this: "I want to take the block that is passed to this method and turn it into a Proc." Just drop the &, and you'll be left with a Proc again:

```
def my_method(&the_proc)
  the_proc
end

p = my_method {|name| "Hello, #{name}!" }
p.class         # => Proc
p.call("Bill")  # => "Hello, Bill!"
```

You now know a bunch of different ways to convert a block to a Proc. But what if you want to convert it back? Again, you can use the & operator to convert the Proc to a block:

blocks/proc_to_block.rb

```
def my_method(greeting)
  "#{greeting}, #{yield}!"
end

my_proc = proc { "Bill" }
my_method("Hello", &my_proc)
```

When you call my_method, the & converts my_proc to a block and passes that block to the method.